

Notebook de obtención, limpieza y transformación de datos — Fase 2

Proyecto: Factores sociodemográficos y riesgo cardiovascular (ENS 2016-2017)

¿Qué es un *pipeline* de preprocesamiento?

Un *pipeline* es una secuencia ordenada de pasos por los que pasan los datos crudos hasta quedar listos para analizar. En este notebook el flujo es:

Obtener → Explorar → Códigos especiales → Limpiar → Transformar → Escalar → Validar

Cada paso se implementa como una **función** reutilizable. Trabajar con funciones (en vez de copiar y pegar código) hace el proceso más ordenado, más fácil de corregir y permite reutilizar la misma lógica sobre cualquier columna sin reescribirla.

Introducción

Las enfermedades cardiovasculares (ECV) son la principal causa de muerte a nivel mundial: en 2023 provocaron 19,2 millones de fallecimientos, una de cada tres muertes globales, cifra que casi se ha duplicado desde los 13,1 millones registrados en 1990 (Global Burden of Disease, 2023). A esto se suma una carga considerable de discapacidad (437 millones de años de vida ajustados por discapacidad) y una prevalencia de hipertensión arterial que afecta a 1.280 millones de adultos entre 30 y 79 años en el mundo, de los cuales solo uno de cada cinco mantiene su condición controlada (OMS, 2023).

Chile no escapa a esta tendencia. Según la Encuesta Nacional de Salud (ENS) 2016-2017 del Ministerio de Salud, la prevalencia nacional de hipertensión arterial alcanza 27,6% y la de sospecha de diabetes mellitus 12,3% en personas de 15 años y más (MINSAL, 2017). Ambas cifras se ubican entre las más altas del conjunto de indicadores medidos por la encuesta, y sitúan a las ECV y sus factores de riesgo como una de las principales prioridades de salud pública del país.

Sin embargo, estas cifras agregadas no informan cómo se distribuye el riesgo al interior de la población. Este proyecto busca avanzar en esa dirección: investigar cómo se asocian las características sociodemográficas de las personas encuestadas —edad, sexo, educación, ingreso del hogar y zona de residencia— con los principales indicadores de riesgo cardiovascular disponibles en la ENS 2016-2017, aportando

evidencia que permita comprender si el riesgo se concentra en subgrupos específicos de la población chilena.

Objetivo general (Fase 2). Construir un *pipeline* reproducible que deje el subconjunto seleccionado en Fase 1 limpio, codificado y escalado, listo para el análisis de asociaciones de las fases siguientes.

Objetivos específicos.

- Obtener el subconjunto de trabajo **siempre a partir del archivo original** `ens2016.xlsx`, seleccionando las 16 variables definidas en la Fase 1.
- Explorar la calidad de los datos y declarar el rol analítico de cada variable.
- Identificar y tratar los códigos especiales de no respuesta del libro de códigos ENS.
- Limpiar gestionando rigurosamente los valores nulos.
- Transformar las variables categóricas (codificación binaria/One-Hot) respetando su naturaleza nominal u ordinal.
- Estandarizar las variables continuas.
- Validar técnicamente el resultado y dejarlo disponible para la Fase 3.

Variables (16): `IdEncuesta`, `FechaInicioF1`, `Edad`, `Sexo`, `Zona`, `HTA`, `di3`, `dis2`, `IMC`, `anos_estudio_MINSAL_1`, `GPAQ`, `as27`, `as28`, `Fexp_F1F2p_Corr`, `Conglomerado`, `Estrato`.

Sobre la variable objetivo. A diferencia de un problema de clasificación clásico (con un único *target* binario), este proyecto es de tipo **asociativo**: no hay una sola variable objetivo, sino varios indicadores de riesgo cardiovascular — `HTA` (hipertensión), `di3` (diabetes), `dis2` (colesterol alto), `IMC` y `GPAQ` (actividad física)— que se analizarán como resultados en las fases posteriores. Este notebook no descarta ninguno de ellos: los dos primeros (`HTA`, `di3`, `dis2`) se limpian y codifican igual que cualquier otra variable, sin transformarlos en un *score* único.

Librerías utilizadas

Librería	Para qué la usamos
<code>numpy</code>	Cálculo numérico y manejo de arreglos.
<code>pandas</code>	Cargar y manipular la tabla de datos (el <code>DataFrame</code>).
<code>matplotlib</code>	Generar los gráficos (<i>boxplots</i>).
<code>sklearn.preprocessing</code>	Codificación (<code>LabelEncoder</code> , <code>OneHotEncoder</code>) y escalamiento (<code>StandardScaler</code> , <code>MinMaxScaler</code> , <code>RobustScaler</code>).

`np.random.seed(2026)` fija la semilla aleatoria —la misma que usa el cuaderno de la Fase 1— para que cualquier proceso con azar dé **siempre el mismo resultado**.

```
In [1]: import numpy as np                # calculo numerico y operaciones vectoriales
import pandas as pd                    # estructuras tabulares: Series y DataFrames
import matplotlib.pyplot as plt       # graficos de control
from sklearn import preprocessing     # LabelEncoder y OneHotEncoder
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

# Reproducibilidad: misma semilla que el cuaderno de la Fase 1
SEMILLA = 2026
np.random.seed(SEMILLA)

%matplotlib inline
```

Configuración del entorno y de las rutas

Se reutiliza el mismo criterio que en la Fase 1: la raíz del proyecto se ubica buscando la carpeta `.git`, no adivinando cuántos niveles subir desde `F2/notebooks`. Así el cuaderno corre igual sin importar desde qué carpeta lo abra Jupyter, y las rutas quedan relativas al repositorio, no al computador de quien lo ejecuta.

```
In [2]: from pathlib import Path
import sys

print("Python:", sys.version.split()[0])
for lib, mod in [("numpy", np), ("pandas", pd)]:
    print(f"{lib:8}:", mod.__version__)

def encontrar_raiz_proyecto(marcador=".git") -> Path:
    """Sube por las carpetas padre hasta encontrar la raíz del repositorio."""
    actual = Path.cwd()
    for carpeta in [actual, *actual.parents]:
        if (carpeta / marcador).exists():
            return carpeta
    raise FileNotFoundError(f"No se encontro '{marcador}' en ningun directorio")

RAIZ = encontrar_raiz_proyecto()
print("\nRaíz del proyecto detectada en:", RAIZ)

DIR_CRUDO = RAIZ / "data" / "raw"           # datos originales: nunca se modifican
DIR_PROCESADO = RAIZ / "data" / "processed" # resultado del pipeline
DIR_DOCS = RAIZ / "F2" / "docs"            # diccionario, resúmenes y metadatos
for carpeta in (DIR_CRUDO, DIR_PROCESADO, DIR_DOCS):
    carpeta.mkdir(parents=True, exist_ok=True)

ARCHIVO_ORIGEN = DIR_CRUDO / "ens2016.xlsx"
print("\nArchivo origen esperado en:", ARCHIVO_ORIGEN)
print("¿Existe?", ARCHIVO_ORIGEN.exists())
```

Python: 3.13.7
numpy : 2.5.3
pandas : 3.0.5

Raíz del proyecto detectada en: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500

Archivo origen esperado en: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500/data/raw/ens2016.xlsx
¿Existe? True

Procedencia del conjunto de datos

Campo	Valor
Título	Encuesta Nacional de Salud (ENS) 2016–2017
Autor / responsable	Ministerio de Salud de Chile (MINSAL), Departamento de Epidemiología
Plataforma	MINSAL — Repositorio de encuestas de salud
Estructura del archivo original	6.233 filas × 1.173 columnas
Subconjunto de trabajo (Fase 1)	16 variables seleccionadas, filtradas por ponderador válido
Unidad de observación	Persona encuestada residente en Chile

Referencia en APA 7 para el informe:

Ministerio de Salud de Chile (MINSAL), Departamento de Epidemiología.
(2017). *Encuesta Nacional de Salud 2016–2017* [Conjunto de datos].
Gobierno de Chile.

Documentar la procedencia no es un trámite: sin ella, nadie puede verificar sobre qué versión del archivo se trabajó.

1. Obtención de los datos

Qué hace este paso. A diferencia de leer directamente un archivo ya recortado, aquí la base **siempre** es `data/raw/ens2016.xlsx` sin modificar. De ese archivo se seleccionan las 16 variables que se definieron y justificaron en la Fase 1, y luego se conserva únicamente a las personas con un ponderador `Fexp_F1F2p_Corr` válido (numérico, mayor que cero, sin infinitos ni vacíos) — el mismo criterio de elegibilidad F1–F2 usado en la Fase 1.

Por qué con funciones. `seleccionar_variables_ens` valida que las 16 columnas existan antes de recortar la tabla (si faltara una, se detiene con un mensaje claro en vez de fallar más adelante de forma confusa). `filtrar_ponderador_valido` aísla la

regla de elegibilidad en un solo lugar, para no repetirla ni describirla de memoria en el informe.

```
In [3]: VARIABLES = [
    "IdEncuesta", "FechaInicioF1", "Edad", "Sexo", "Zona", "HTA", "di3", "di4",
    "IMC", "anos_estudio_MINSAL_1", "GPAQ", "as27", "as28", "Fexp_F1F2p_Corr",
    "Conglomerado", "Estrato",
]

def seleccionar_variables_ens(ruta_origen, variables):
    """
    Lee la base origen de la ENS y recorta las columnas indicadas.

    No cambia codigos ni excluye registros: es una seleccion de COLUMNAS,
    coherente con la caracterizacion de variables hecha en la Fase 1.

    Lanza
    -----
    FileNotFoundError
        Si el archivo origen no existe.
    ValueError
        Si falta alguna variable esperada en la base origen.
    """
    if not Path(ruta_origen).exists():
        raise FileNotFoundError(f"No se encontro la base origen en:\n{ruta_origen}")

    base = pd.read_excel(ruta_origen, sheet_name="Sheet1")
    faltantes = [v for v in variables if v not in base.columns]
    if faltantes:
        raise ValueError(f"No se encontraron estas variables en la base origen: {faltantes}")

    print(f"Base origen: {base.shape[0]} filas x {base.shape[1]} columnas")
    return base[variables].copy()

seleccion = seleccionar_variables_ens(ARCHIVO_ORIGEN, VARIABLES)
print(f"Seleccion (16 variables): {seleccion.shape[0]} filas x {seleccion.shape[1]} columnas")

ARCHIVO_SELECCIONADAS = DIR_PROCESADO / "ens_variables_seleccionadas.xlsx"
seleccion.to_excel(ARCHIVO_SELECCIONADAS, index=False)
print(f"Guardado: {ARCHIVO_SELECCIONADAS}")
```

Base origen: 6233 filas x 1173 columnas

Seleccion (16 variables): 6233 filas x 16 columnas

Guardado: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500/data/processed/ens_variables_seleccionadas.xlsx

Con las 16 variables recortadas, se aplica el filtro de elegibilidad F1–F2: no toda persona encuestada en la Fase 1 fue re-contactada en la Fase 2, y quienes no lo fueron no tienen un ponderador combinado válido para analizarlas en conjunto.

```
In [4]: def filtrar_ponderador_valido(df, columna_peso):
    """
```

Conserva solo las filas con ponderador valido: numerico, > 0, finito y r

Retorna

```
tuple(pd.DataFrame, pd.DataFrame)
    El subconjunto filtrado y una tabla resumen del filtro (trazabilidad)
"""
peso = pd.to_numeric(df[columna_peso], errors="coerce")
valido = peso.notna() & peso.gt(0) & peso.lt(float("inf"))

filtrado = df.loc[valido].copy()
filtrado[columna_peso] = peso.loc[valido]

resumen = pd.DataFrame({
    "indicador": [
        "Personas antes del filtro",
        "Personas con ponderador valido",
        "Personas excluidas por ponderador no valido",
        "Suma de ponderadores del subconjunto",
    ],
    "valor": [
        len(df), len(filtrado), len(df) - len(filtrado), filtrado[columna_peso].sum()
    ],
})
return filtrado, resumen

seleccion_f1f2, resumen_filtro = filtrar_ponderador_valido(seleccion, "Fexp_

ARCHIVO_F1F2 = DIR_PROCESADO / "ens_variables_f1f2.xlsx"
seleccion_f1f2.to_excel(ARCHIVO_F1F2, index=False)
resumen_filtro.to_csv(DIR_DOCS / "resumen_filtro_f1f2.csv", index=False)

print(f"Guardado: {ARCHIVO_F1F2}")
resumen_filtro
```

Guardado: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500/data/processed/ens_variables_f1f2.xlsx

Out[4]:

	indicador	valor
0	Personas antes del filtro	6.233000e+03
1	Personas con ponderador valido	5.520000e+03
2	Personas excluidas por ponderador no valido	7.130000e+02
3	Suma de ponderadores del subconjunto	1.451897e+07

Trazabilidad con la Fase 1. El cuaderno

F1/notebooks/S1_F1_Definicion.ipynb reconoce y verifica este mismo archivo (data/processed/ens_variables_f1f2.xlsx) al auditar la estructura del repositorio. Regenerarlo aquí, siempre desde ens2016.xlsx , evita que ambas fases queden documentando un archivo que alguien pudo haber editado a mano en el camino.

```
In [5]: # df_crudo es el punto de partida SIN limpiar: se usa mas adelante para comp
# el efecto de cada transformacion contra el estado original.
df_crudo = seleccion_f1f2.reset_index(drop=True)
df = df_crudo.copy()

print(f"Filas: {df.shape[0]}")
print(f"Columnas: {df.shape[1]}")
df.head()
```

Filas: 5520

Columnas: 16

```
Out [5]:
```

	IdEncuesta	FechaInicioF1	Edad	Sexo	Zona	HTA	di3	dis2	IMC	anos_es
--	------------	---------------	------	------	------	-----	-----	------	-----	---------

0	20008	2016-11-27 15:10:40	25	2	1	0.0	2	3	23.198208	
---	-------	------------------------	----	---	---	-----	---	---	-----------	--

1	20011	2016-11-26 15:52:49	20	2	1	0.0	2	3	26.203983	
---	-------	------------------------	----	---	---	-----	---	---	-----------	--

2	20012	2016-11-20 13:16:39	85	2	1	1.0	2	2	23.732346	
---	-------	------------------------	----	---	---	-----	---	---	-----------	--

3	20017	2016-12-02 15:25:24	30	2	1	0.0	2	3	25.015243	
---	-------	------------------------	----	---	---	-----	---	---	-----------	--

4	20018	2016-11-26 16:02:05	53	2	1	0.0	2	3	32.290249	
---	-------	------------------------	----	---	---	-----	---	---	-----------	--

2. Exploración inicial

Qué hace este paso. Antes de modificar nada, miramos el estado original de los datos.

Esto da la "línea base" contra la cual se verifica después cada transformación.

La función `explorar_dataframe` reporta dimensiones, tipos de datos, nulos por columna y estadísticos descriptivos de las variables numéricas.

```
In [6]: def explorar_dataframe(datos):
        """Resumen exploratorio: dimensiones, tipos, nulos y estadisticos descri
        print("Dimensiones:", datos.shape)
        print("\nTipos de datos:")
        print(datos.dtypes)
        print("\nValores nulos por columna:")
        print(datos.isnull().sum())
        print("\nEstadisticos descriptivos (variables numericas):")
        return datos.describe()

        explorar_dataframe(df)
```

Dimensiones: (5520, 16)

Tipos de datos:

IdEncuesta	int64
FechaInicioF1	datetime64[us]
Edad	int64
Sexo	int64
Zona	int64
HTA	float64
di3	int64
dis2	int64
IMC	float64
anos_estudio_MINSAL_1	float64
GPAQ	float64
as27	float64
as28	int64
Fexp_F1F2p_Corr	float64
Conglomerado	int64
Estrato	int64
dtype:	object

Valores nulos por columna:

IdEncuesta	0
FechaInicioF1	0
Edad	0
Sexo	0
Zona	0
HTA	9
di3	0
dis2	0
IMC	37
anos_estudio_MINSAL_1	47
GPAQ	196
as27	995
as28	0
Fexp_F1F2p_Corr	0
Conglomerado	0
Estrato	0
dtype:	int64

Estadisticos descriptivos (variables numericas):

Out [6]:

	IdEncuesta	FechaInicioF1	Edad	Sexo	Zona	
count	5520.000000	5520	5520.000000	5520.000000	5520.000000	5511.
mean	25781.190580	2016-11-07 22:08:31.675181	49.202536	1.634239	1.158514	0
min	20008.000000	2016-08-04 12:16:22	15.000000	1.000000	1.000000	0.
25%	22672.750000	2016-10-14 12:26:29.250000	33.000000	1.000000	1.000000	0.
50%	25751.500000	2016-11-05 15:50:20	50.000000	2.000000	1.000000	0.
75%	28504.250000	2016-11-27 15:56:07	64.000000	2.000000	1.000000	1.
max	70000.000000	2017-02-23 18:03:30	98.000000	2.000000	2.000000	1.
std	3685.647171	NaN	19.266931	0.481687	0.365256	0.

También revisamos **cuántas categorías distintas** tiene cada variable candidata a nominal, ordinal o binaria, y cuántas veces aparece cada una. `dropna=False` incluye los nulos como una categoría más: si una variable tiene faltantes se quiere verlos aquí, no que desaparezcan del conteo.

```
In [7]: columnas_categoricas = ["Sexo", "Zona", "di3", "dis2", "GPAQ", "as28"]
for col in columnas_categoricas:
    print(f"\n{col}:")
    print(df[col].value_counts(dropna=False).sort_index())
```

```
Sexo:
Sexo
1      2019
2      3501
Name: count, dtype: int64
```

```
Zona:
Zona
1      4645
2       875
Name: count, dtype: int64
```

```
di3:
di3
1       816
2      4653
3        51
Name: count, dtype: int64
```

```
dis2:
dis2
1       767
2       646
3      3836
4       271
Name: count, dtype: int64
```

```
GPAQ:
GPAQ
1.0      2206
2.0      1224
3.0      1894
NaN       196
Name: count, dtype: int64
```

```
as28:
as28
-9999      818
1         102
2         421
3         752
4         641
5         782
6         518
7         634
8         206
9         364
10        142
11        140
Name: count, dtype: int64
```

El diccionario de variables

Antes de limpiar hay que declarar **qué es cada variable**. En este conjunto, `Sexo`, `di3` e `IdEncuesta` son todas numéricas y sin embargo exigen tratamientos

completamente distintos: el rol analítico no lo dice el `dtype`, lo decide el equipo.

Rol analítico	Qué preprocesamiento exige
Continua	Detección de atípicos, imputación, escalamiento
Discreta	Imputación; no siempre requiere escalamiento
Binaria	Codificación 0/1 y revisión de desbalance
Nominal	<i>One-hot encoding</i>
Ordinal	Se conserva el orden — no se aplica <i>one-hot</i>
Identificador	Verificar unicidad y excluir del análisis
Fecha	Se documenta; se excluye de la matriz de variables explicativas

Los roles de la tabla siguiente son los que **el equipo declaró** en la Fase 1

(`CLASIFICACION_EQUIPO`, contrastado ya contra el validador automático en `docs/informe_validacion_f1f2.md`), con una excepción documentada más abajo: `GPAQ` se reclasifica de *nominal* a *ordinal* en este cuaderno.

```
In [8]: # Rol asignado por el equipo en la Fase 1 (ver docs/informe_validacion_f1f2.md)
# GPAQ se corrige aquí de "nominal" a "ordinal": el nivel de actividad física
# (Bajo < Moderado < Alto) tiene un orden con sentido en el dominio, y trata
# como nominal perdería esa información al codificarlo. Ver sección 5.
DECLARADO = [
    ("IdEncuesta",          "identificador", "Identificador único de la encuesta"),
    ("FechaInicioF1",      "fecha",         "Fecha y hora de inicio de la encuesta"),
    ("Edad",                "continua",      "Edad en años"),
    ("Sexo",                "binaria",       "1 = Hombre, 2 = Mujer"),
    ("Zona",                "binaria",       "1 = Urbano, 2 = Rural"),
    ("HTA",                 "binaria",       "1 = hipertensión arterial controlada"),
    ("di3",                 "nominal",       "Diagnóstico de diabetes (controlada)"),
    ("dis2",                 "nominal",       "Diagnóstico de colesterol alto"),
    ("IMC",                 "continua",      "Índice de masa corporal"),
    ("anos_estudio_MINSAL_1", "discreta",    "Años de escolaridad"),
    ("GPAQ",                 "ordinal",      "Nivel de actividad física: Bajo, Moderado, Alto"),
    ("as27",                 "continua",      "Ingreso total del hogar (per cápita)"),
    ("as28",                 "ordinal",      "Tramo de ingreso per cápita"),
    ("Fexp_F1F2p_Corr",     "continua",      "Ponderador muestral combinado"),
    ("Conglomerado",        "identificador", "Conglomerado del diseño muestral"),
    ("Estrato",              "identificador", "Estrato del diseño muestral")
]
diccionario = pd.DataFrame(DECLARADO, columns=["variable", "rol", "descripción"])

# Columnas OBSERVADAS: se leen del archivo, no se escriben a mano
diccionario["dtype"] = [str(df_crudo[v].dtype) for v in diccionario["variable"]]
diccionario["n_unicos"] = [int(df_crudo[v].nunique()) for v in diccionario["variable"]]
diccionario["pct_nulos"] = [round(df_crudo[v].isna().mean() * 100, 1) for v in diccionario["variable"]]

diccionario.to_csv(DIR_DOCS / "diccionario_variables.csv", index=False)
diccionario
```

Out [8]:

	variable	rol	descripcion	dtype	n_unicos	pct_i
0	IdEncuesta	identificador	Identificador unico de la persona encuestada	int64	5520	
1	FechalnicioF1	fecha	Fecha y hora de inicio de la entrevista F1	datetime64[us]	5512	
2	Edad	continua	Edad en años	int64	83	
3	Sexo	binaria	1 = Hombre, 2 = Mujer	int64	2	
4	Zona	binaria	1 = Urbano, 2 = Rural	int64	2	
5	HTA	binaria	1 = hipertension arterial diagnosticada	float64	2	
6	di3	nominal	Diagnostico de diabetes (codigos 1/2/3, ver li...	int64	3	
7	dis2	nominal	Diagnostico de colesterol alto (codigos 1 a 4,...	int64	4	
8	IMC	continua	Indice de masa corporal	float64	5332	
9	anos_estudio_MINSAL_1	discreta	Años de escolaridad	float64	23	
10	GPAQ	ordinal	Nivel de actividad fisica: 1 Bajo < 2 Moderado...	float64	3	
11	as27	continua	Ingreso total del hogar (pesos chilenos)	float64	254	
12	as28	ordinal	Tramo de ingreso per capita (1 a 11, orden cre...	int64	12	
13	Fexp_F1F2p_Corr	continua	Ponderador muestral combinado F1-F2 (diseno, n...	float64	4764	

	variable	rol	descripcion	dtype	n_unicos	pct_i
14	Conglomerado	identificador	Conglomerado del diseno muestral (diseno, no p...	int64	1075	
15	Estrato	identificador	Estrato del diseno muestral (diseno, no predic...	int64	30	

Sobre di3 y dis2. Sus categorías minoritarias no son códigos de no respuesta (-8888 / -9999) — eso se verifica en la sección siguiente —, sino respuestas válidas según el libro de códigos F1 de la ENS 2016–2017: `di3 = 3` es "No recuerda" y `dis2 = 4` es "No recuerda" (`dis2 = 3` significa "Nunca", una respuesta negativa válida, no incertidumbre). Se conservan como categorías propias del one-hot (sección 5), con su nombre real, en vez de fusionarlas con "No"/"Nunca" o de descartarlas.

Valores atípicos: medir antes de decidir

Se usa el criterio del rango intercuartílico sobre las tres variables continuas (`Edad` , `IMC` , `as27`):

$$\text{atípico si } x < Q_1 - 1,5 \cdot \text{RIC} \quad \text{o} \quad x > Q_3 + 1,5 \cdot \text{RIC}$$

```
In [9]: def detectar_atipicos_iqr(serie, factor=1.5):
        """Identifica valores atipicos por el criterio del rango intercuartilico

        Retorna
        -----
        tuple(np.ndarray, float, float)
            Mascara booleana de atipicos, limite inferior y limite superior.
        """
        valores = serie.dropna().to_numpy(dtype="float64")
        q1, q3 = np.percentile(valores, [25, 75])
        ric = q3 - q1
        limite_inf, limite_sup = q1 - factor * ric, q3 + factor * ric

        mascara = (serie < limite_inf) | (serie > limite_sup)
        return mascara.fillna(False).to_numpy(), limite_inf, limite_sup

filas = []
for col in ["Edad", "IMC", "as27"]:
    mascara, inf, sup = detectar_atipicos_iqr(df[col])
    filas.append({
        "variable": col,
```

```

        "limite_inf": round(Inf, 2),
        "limite_sup": round(sup, 2),
        "n_atipicos": int(mascara.sum()),
        "pct_atipicos": round(mascara.mean() * 100, 2),
        "media": round(df[col].mean(), 2),
        "mediana": round(df[col].median(), 2),
    })
pd.DataFrame(filas)

```

Out[9]:

	variable	limite_inf	limite_sup	n_atipicos	pct_atipicos	media	mediana
0	Edad	-13.50	110.50	0	0.00	49.20	50.00
1	IMC	14.64	42.44	115	2.08	28.93	28.32
2	as27	-250000.00	950000.00	405	7.34	469382.67	300000.00

Cómo se lee esta tabla. En `as27` (ingreso del hogar) la media queda muy por encima de la mediana: unos pocos hogares con ingresos extremos arrastran el promedio. Ese es el argumento técnico para imputar con la mediana en la sección de limpieza, no una preferencia por defecto.

3. Códigos especiales de no respuesta

Antes de imputar nada hay que distinguir un **valor faltante real** (`NaN`) de un **código de no respuesta**: un entero válido desde el punto de vista de pandas (`-8888` o `-9999`) que en el libro de códigos F1/F2 de la ENS significa "No sabe" o "No responde" respectivamente. Si no se detectan, `pandas.isna()` los deja pasar como si fueran datos numéricos legítimos y contaminan cualquier estadístico o imputación posterior.

Se revisan las seis variables candidatas — `as27`, `as28`, `di3`, `dis2`, `HTA`, `GPAQ` — **sin asumir que todas usan el mismo esquema**: cada una se cuenta por separado.

```

In [10]: def revisar_codigos_especiales(df, columnas, codigos=(-8888, -9999)):
    """Cuenta cuantas veces aparece cada codigo especial en cada columna."""
    filas = []
    for col in columnas:
        conteo = {"variable": col}
        for cod in codigos:
            conteo[f"n_{cod}"] = int((df[col] == cod).sum())
        filas.append(conteo)
    return pd.DataFrame(filas)

variables_a_revisar = ["as27", "as28", "di3", "dis2", "HTA", "GPAQ"]
revisar_codigos_especiales(df, variables_a_revisar)

```

Out[10]:

	variable	n_-8888	n_-9999
0	as27	0	0
1	as28	0	818
2	di3	0	0
3	dis2	0	0
4	HTA	0	0
5	GPAQ	0	0

Resultado. De las seis variables candidatas, solo `as28` contiene efectivamente el código `-9999` (no aparece `-8888` en ninguna). Es una comprobación, no un supuesto: las demás variables no necesitan este tratamiento, y aplicárselo igual sin verificar habría sido un paso de limpieza injustificado.

Se reemplaza `-9999` por `NaN` en `as28` y, como el hecho de no responder puede ser informativo en sí mismo (a diferencia de un olvido aleatorio), se conserva en una columna aparte antes de imputar — el mismo criterio que la bandera `bmi_imputado` del patrón visto en el curso.

```
In [11]: def marcar_codigos_no_respuesta(df, columna, codigos=(-8888, -9999)):
        """
        Reemplaza los codigos de no respuesta indicados por NaN en una columna.

        Retorna
        -----
        pd.DataFrame
            Copia del DataFrame con la columna corregida.
        """
        if columna not in df.columns:
            raise KeyError(f"La columna '{columna}' no existe en el DataFrame.")
        df = df.copy()
        es_codigo = df[columna].isin(codigos)
        n = int(es_codigo.sum())
        df.loc[es_codigo, columna] = np.nan
        print(f"'{columna}': {n} valores con codigo de no respuesta marcados con")
        return df

df["as28_no_responde"] = df["as28"].isin([-8888, -9999]).astype(int)
df = marcar_codigos_no_respuesta(df, "as28")
```

'as28': 818 valores con codigo de no respuesta marcados como NaN

4. Limpieza de datos

Se toman seis decisiones, cada una justificada con las cifras ya obtenidas:

a) Excluir identificadores. `IdEncuesta` es un identificador único de persona: no aporta información predictiva. `Conglomerado` y `Estrato` son variables de **diseño muestral** (identifican el conglomerado y estrato de muestreo), no predictoras — se conservan en el archivo final para que la Fase 3 pueda usarlas en un análisis con diseño muestral, pero quedan fuera de cualquier transformación (no se codifican ni escalan). `FechaInicioF1` (fecha) tampoco es una variable explicativa del riesgo cardiovascular en este alcance, así que se excluye de la matriz que se transforma más abajo.

b) Eliminar las filas con HTA nulo. Son 9 personas (0,16 % de la muestra) sin diagnóstico de hipertensión registrado. A diferencia de `bmi` en el ejemplo del curso, `HTA` es un diagnóstico clínico binario: imputarlo (con la moda, que sería "No") equivale a afirmar que esas 9 personas no tienen hipertensión sin que nadie lo haya reportado. Con una fracción tan pequeña de la muestra, eliminar las filas es más defendible que inventar un diagnóstico.

c) Imputar IMC y anos_estudio_MINSAL_1 con la mediana simple; as27 con un tratamiento dividido (detalle en la sección 2). `IMC` y `anos_estudio_MINSAL_1` tienen nulos reales con distribuciones asimétricas y se imputan con la mediana global. `as27` es distinto: 816 de sus nulos (en el conjunto de trabajo, ya sin las 9 filas de la decisión b) son la misma no respuesta económica de `as28` (decisión e) y no se imputan; los 177 restantes sí declararon su tramo de ingreso y se imputan con la mediana de `as27` dentro de ese tramo.

d) Imputar GPAQ con la moda. Para una variable ordinal, la moda conserva una categoría observada real en la escala, a diferencia de una imputación numérica tipo mediana. Este criterio no se aplica igual a todas las variables ordinales con nulos: se evalúa caso a caso según qué tan verificable es el patrón de no respuesta y qué tan costoso sería imputar mal (ver la decisión sobre `as28` a continuación, que sigue un criterio distinto). Para `GPAQ` específicamente no hay evidencia verificada de un patrón de no respuesta — solo la duda razonable sobre su procedencia, ya que no aparece documentado como variable cruda en el libro de códigos y es probablemente un índice ya calculado por MINSAL según el protocolo GPAQ de la OMS — y el costo de imputar mal es bajo: apenas 3,6 % de nulos (196 de 5.520), con `GPAQ = 1` (Bajo) ya como categoría mayoritaria (41,4 %) entre quienes sí respondieron.

e) No imputar as28. El conjunto elegible tenía 818 registros con código `-9999` ("No responde") en `as28`; 816 de ellos siguen en el conjunto de trabajo tras excluir las 9 filas de la decisión b. Ninguno de esos 816 tiene un valor válido en `as27` (monto de ingreso del hogar): son exactamente los mismos casos, no dos fallas independientes. Quien no informa el monto tampoco informa el tramo — es la misma no respuesta económica, con un patrón que podría no ser aleatorio (concentrado en ingresos muy altos o muy bajos). Imputar con la moda escondería ese posible sesgo bajo un valor típico inventado. Se deja como faltante genuino (`NaN`) para que la Fase 3 decida su tratamiento con pleno conocimiento de esta limitación; la no respuesta ya queda

registrada aparte en `as28_no_responde` (sección 3), sin necesidad de tocar la escala ordinal de `as28`.

f) `di3`, `dis2`, `Sexo`, `Zona` no tienen nulos reales: no requieren imputación.

```
In [12]: def imputar_nulos_numericos(df, columna, estrategia="mediana"):
        """
        Imputa los valores nulos de una columna numerica continua o discreta.

        Parametros
        -----
        df : pd.DataFrame
        columna : str
        estrategia : str
                'media' o 'mediana'.

        Retorna
        -----
        pd.DataFrame
        """
        if columna not in df.columns:
            raise KeyError(f"La columna '{columna}' no existe en el DataFrame.")
        n_nulos = int(df[columna].isnull().sum())
        if estrategia == "media":
            valor = df[columna].mean()
        elif estrategia == "mediana":
            valor = df[columna].median()
        else:
            raise ValueError("estrategia debe ser 'media' o 'mediana'")
        df = df.copy()
        df[columna] = df[columna].fillna(valor)
        print(f"'{columna}': {n_nulos} nulos imputados con la {estrategia} = {valor}")
        return df

def imputar_nulos_categoricos(df, columna, estrategia="moda"):
    """
    Imputa los valores nulos de una columna categorica (nominal u ordinal) con la moda.

    Retorna
    -----
    pd.DataFrame
    """
    if columna not in df.columns:
        raise KeyError(f"La columna '{columna}' no existe en el DataFrame.")
    if estrategia != "moda":
        raise ValueError("estrategia debe ser 'moda'")
    n_nulos = int(df[columna].isnull().sum())
    valor = df[columna].mode(dropna=True).iloc[0]
    df = df.copy()
    df[columna] = df[columna].fillna(valor)
    print(f"'{columna}': {n_nulos} nulos imputados con la moda = {valor}")
    return df
```

Antes de limpiar, confirmamos dónde están los nulos:

```
In [13]: df.isnull().sum()
```

```
Out[13]: IdEncuesta          0
FechaInicioF1          0
Edad                   0
Sexo                   0
Zona                   0
HTA                    9
di3                    0
dis2                   0
IMC                   37
anos_estudio_MINSAL_1  47
GPAQ                  196
as27                  995
as28                  818
Fexp_F1F2p_Corr        0
Conglomerado           0
Estrato                0
as28_no_responde        0
dtype: int64
```

Excluimos el identificador `IdEncuesta` de la matriz de variables (se verifica antes que sea efectivamente único):

```
In [14]: assert df["IdEncuesta"].is_unique, "'IdEncuesta' tiene valores repetidos: re
df = df.drop(columns=["IdEncuesta", "FechaInicioF1"])
df.head()
```

```
Out[14]:
```

	Edad	Sexo	Zona	HTA	di3	dis2	IMC	anos_estudio_MINSAL_1	GPAQ	
0	25	2	1	0.0	2	3	23.198208	15.0	3.0	20
1	20	2	1	0.0	2	3	26.203983	14.0	3.0	
2	85	2	1	1.0	2	2	23.732346	15.0	1.0	130
3	30	2	1	0.0	2	3	25.015243	16.0	3.0	
4	53	2	1	0.0	2	3	32.290249	14.0	1.0	

Eliminamos las filas con `HTA` nulo:

```
In [15]: n_antes = len(df)
df = df.dropna(subset=["HTA"]).reset_index(drop=True)
print(f"Filas eliminadas por 'HTA' nulo: {n_antes - len(df)}")
print(f"Filas restantes: {len(df)}")
```

Filas eliminadas por 'HTA' nulo: 9
Filas restantes: 5511

Imputamos `IMC` con la mediana:

```
In [16]: df = imputar_nulos_numericos(df, "IMC", estrategia="mediana")
```

'IMC': 37 nulos imputados con la mediana = 28.31

Comparar los métodos antes de elegir en as27

as27 (ingreso del hogar) es la variable con más nulos reales (18 %) y con los atípicos más extremos detectados en la sección 2. La función siguiente compara cuatro estrategias, incluida una imputación por grupo de edad, antes de aplicar una.

```
In [17]: def comparar_imputaciones(datos, columna, agrupador=None, n_grupos=4):
    """Compara el efecto de varias estrategias de imputacion sobre una variable.

    Retorna
    -----
    pd.DataFrame con el efecto de cada estrategia sobre la distribucion.
    """
    original = datos[columna]
    if original.isna().sum() == 0:
        raise ValueError(f"'{columna}' no tiene valores faltantes.")

    resultados = {
        "sin imputar (referencia)": original.dropna(),
        "eliminar filas": original.dropna(),
        "media": original.fillna(original.mean()),
        "mediana": original.fillna(original.median()),
    }
    if agrupador is not None:
        tramos = pd.qcut(datos[agrupador], q=n_grupos, duplicates="drop")
        resultados["mediana por grupo"] = original.fillna(
            original.groupby(tramos, observed=True).transform("median"))

    filas = [{
        "estrategia": nombre,
        "n": len(serie.dropna()),
        "media": round(serie.mean(), 3),
        "desv_est": round(serie.std(), 3),
        "asimetria": round(serie.skew(), 3),
    } for nombre, serie in resultados.items()]

    tabla = pd.DataFrame(filas)
    referencia = tabla.iloc[0]["desv_est"]
    tabla["cambio_desv_%"] = ((tabla["desv_est"] - referencia) / referencia)
    tabla["filas_perdidas"] = len(datos) - tabla["n"]
    return tabla
```

```
comparar_imputaciones(df_crudo, "as27", agrupador="Edad")
```

Out [17]:	estrategia	n	media	desv_est	asimetria	cambio_desv_%	filas_perdi
0	sin imputar (referencia)	4525	469382.671	624183.272	11.315	0.00	9
1	eliminar filas	4525	469382.671	624183.272	11.315	0.00	9
2	media	5520	469382.671	565123.317	12.497	-9.46	
3	mediana	5520	438850.831	568862.464	12.408	-8.86	
4	mediana por grupo	5520	445622.570	568110.839	12.422	-8.98	

El ingreso del hogar tiene una asimetría fuerte (unos pocos hogares con ingresos muy altos), y su no respuesta probablemente no es aleatoria: es plausible que se concentre en los extremos (ingresos muy altos o muy bajos). Por eso `as27` no recibe un único tratamiento: los 816 casos que tampoco respondieron `as28` (la misma no respuesta económica, decisión e de la sección 4) se dejan como faltante genuino, sin inventar un valor. Los 177 casos restantes sí declararon su tramo de ingreso (`as28`), así que se imputan con la **mediana de `as27` dentro de ese mismo tramo** — más preciso que una mediana por grupo de edad, porque usa información que la persona sí entregó. Esto reduce la distorsión frente a imputar con la mediana general o por edad, pero no elimina el posible sesgo de no aleatoriedad: se documenta como limitación para la interpretación de la Fase 3, no como un problema resuelto aquí.

```
In [18]: # as27 tiene dos causas de nulo distintas (ver decision e, seccion 4). Se us
# flag as28_no_responde (seccion 3) en vez de df_crudo: df_crudo no se vuelv
# sincronizar despues del reset_index() al eliminar filas con HTA nulo, y co
# contra el ya no alinearia las filas correctamente.
# - Los que tampoco respondieron as28 (as28_no_responde == 1): no se imputan
# - Los que si respondieron as28 pero no el monto exacto: se imputan con la
#   de as27 dentro de su propio tramo de as28, mas informativo que el grupo
sin_dato_total = df["as28_no_responde"] == 1
falta_solo_monto = df["as27"].isna() & ~sin_dato_total

df["as27_imputado"] = 0
mediana_por_tramo = df["as27"].groupby(df["as28"], observed=True).transform(
df.loc[falta_solo_monto, "as27"] = mediana_por_tramo.loc[falta_solo_monto]
df.loc[falta_solo_monto, "as27_imputado"] = 1

print(f"Filas sin dato total (as27 y as28 nulos, no se imputan): {int(sin_da
print(f"Filas imputadas en 'as27' con mediana por tramo de as28: {int(falta_
print(f"'as27' con nulos restantes (a proposito): {int(df['as27'].isna()).sum

Filas sin dato total (as27 y as28 nulos, no se imputan): 816
Filas imputadas en 'as27' con mediana por tramo de as28: 177
'as27' con nulos restantes (a proposito): 816
```

Fuga de datos.** El valor de relleno se calcula sobre todo el conjunto

porque no hay partición entre entrenamiento y prueba. Cuando la haya, la mediana (por grupo o global) debe calcularse **solo con el conjunto de entrenamiento**. El mismo cuidado aplica al escalamiento de la sección 6.

Imputamos `anos_estudio_MINSAL_1` (discreta) con la mediana y `GPAQ` (ordinal) con la moda. `as28` no se imputa (ver decisión e, sección 4):

```
In [19]: df = imputar_nulos_numericos(df, "anos_estudio_MINSAL_1", estrategia="mediar")
df = imputar_nulos_categoricos(df, "GPAQ", estrategia="moda")
# as28 NO se imputa (ver decision e, seccion 4): queda como NaN a proposito.

'anos_estudio_MINSAL_1': 47 nulos imputados con la mediana = 11.00
'GPAQ': 196 nulos imputados con la moda = 1.0
```

Después de limpiar, verificamos que ya no quede ningún nulo:

```
In [20]: df.isnull().sum()
```

```
Out[20]: Edad                0
Sexo                0
Zona                0
HTA                0
di3                0
dis2                0
IMC                0
anos_estudio_MINSAL_1  0
GPAQ                0
as27                816
as28                816
Fexp_F1F2p_Corr      0
Conglomerado        0
Estrato              0
as28_no_responde     0
as27_imputado        0
dtype: int64
```

5. Transformación

Los modelos de asociación de las fases siguientes necesitan una matriz completamente numérica. Cómo se codifica cada variable depende de su rol, ya declarado en el diccionario:

- `Sexo` y `Zona` son **binarias** con códigos `1 / 2` : se codifican como el resto de las nominales de dos categorías (equivalente a `ever_married` o `Residence_type` en el patrón visto en el curso), quedando cada una en dos columnas de 0 y 1.
- `HTA` **ya está en 0/1**: no necesita codificación adicional.
- `di3` y `dis2` son **nominales** de 3 y 4 categorías: *One-Hot Encoding*.
- `GPAQ` y `as28` son **ordinales** y ya están representadas como enteros en el orden correcto (`GPAQ` : 1 Bajo < 2 Moderado < 3 Alto; `as28` : tramos de ingreso 1 a 11

crecientes): se **dejan como están**, sin *one-hot*, precisamente para no destruir esa información de orden.

Se mantiene el método visto en el curso — `LabelEncoder` seguido de `OneHotEncoder` — encapsulado en una función.

El patrón `fit` / `transform`

Casi todas las herramientas de scikit-learn funcionan en dos tiempos: **`fit`** (aprender qué categorías existen) y **`transform`** (aplicar esa codificación). La función `codificar_one_hot` los encadena para no repetir el mismo bloque en cada variable, y valida que la cantidad de nombres entregados coincida con la cantidad de categorías detectadas, avisando con un error claro si no.

```
In [21]: def codificar_one_hot(df, columna, nombres_columnas):
        """
        Codifica una variable categorica aplicando LabelEncoder y luego OneHotEncoder
        (patron fit/transform de scikit-learn).

        Parametros
        -----
        df : pd.DataFrame
        columna : str
            Columna categorica a codificar.
        nombres_columnas : list[str]
            Nombres de las nuevas columnas binarias. El orden debe coincidir con
            orden ASCENDENTE de las categorias (criterio interno de LabelEncoder)

        Retorna
        -----
        pd.DataFrame

        Lanza
        -----
        KeyError
            Si la columna no existe.
        ValueError
            Si el numero de nombres no coincide con el numero de categorias.
        """
        if columna not in df.columns:
            raise KeyError(f"La columna '{columna}' no existe en el DataFrame.")

        le = preprocessing.LabelEncoder()
        datos = df[columna]
        le.fit(datos)
        datos_codificados = le.transform(datos)

        if len(nombres_columnas) != len(le.classes_):
            raise ValueError(
                f"Se esperaban {len(le.classes_)} nombres para '{columna}' "
                f"({list(le.classes_)}), pero se recibieron {len(nombres_columnas)}"
            )
```

```

ohe = preprocessing.OneHotEncoder()
d = datos_codificados.reshape(-1, 1)
ohe.fit(d)
matriz = ohe.transform(d).toarray()

nuevas = pd.DataFrame(matriz, columns=nombres_columnas, index=df.index).

df = df.drop(columns=[columna]).reset_index(drop=True)
nuevas = nuevas.reset_index(drop=True)
return pd.concat([df, nuevas], axis=1)

```

a. Sexo y Zona (binarias)

Categorías (orden ascendente): Sexo → 1 Hombre, 2 Mujer. Zona → 1 Urbano, 2 Rural.

```

In [22]: df = codificar_one_hot(df, "Sexo", ["Sexo_Hombre", "Sexo_Mujer"])
df = codificar_one_hot(df, "Zona", ["Zona_Urbano", "Zona_Rural"])
df.head()

```

```

Out[22]:

```

	Edad	HTA	di3	dis2	IMC	anos_estudio_MINSAL_1	GPAQ	as27	as28
0	25	0.0	2	3	23.198208	15.0	3.0	200000.0	3.0
1	20	0.0	2	3	26.203983	14.0	3.0	NaN	NaN
2	85	1.0	2	2	23.732346	15.0	1.0	1300000.0	10.0
3	30	0.0	2	3	25.015243	16.0	3.0	NaN	NaN
4	53	0.0	2	3	32.290249	14.0	1.0	NaN	NaN

b. di3 y dis2 (nominales)

Sus categorías se nombran según su significado real, verificado contra el libro de códigos F1 de la ENS 2016–2017: di3 (1=Sí, 2=No, 3=No recuerda) y dis2 (1=Sí una vez, 2=Sí más de una vez, 3=Nunca, 4=No recuerda). Las categorías de "No recuerda" se mantienen separadas de "No"/"Nunca": fusionarlas equivaldría a inventar una respuesta que la persona no dio.

```

In [23]: # Categorías según el libro de códigos oficial F1 de la ENS 2016–2017:
# di3: 1 = Si, 2 = No, 3 = No recuerda
# dis2: 1 = Si una vez, 2 = Si mas de una vez, 3 = Nunca, 4 = No recuerda
#
# "No recuerda" (di3=3, dis2=4) es una categoría de INCERTIDUMBRE, no un
# valor faltante ni una respuesta negativa: no se fusiona con "No" ni con
# "Nunca". Se conserva como su propia columna en el one-hot para que el
# analisis de la Fase 3 pueda decidir como tratarla, en vez de que esta
# decision quede oculta dentro de la codificacion.

```

```
df = codificar_one_hot(df, "di3", ["di3_Si", "di3_No", "di3_No_recuerda"])
df = codificar_one_hot(
    df, "dis2",
    ["dis2_Si_una_vez", "dis2_Si_mas_de_una_vez", "dis2_Nunca", "dis2_No_rec"]
)
df.head()
```

Out [23]:

	Edad	HTA	IMC	anos_estudio_MINSAL_1	GPAQ	as27	as28	Fexp_F1F
0	25	0.0	23.198208		15.0	3.0	200000.0	3.0
1	20	0.0	26.203983		14.0	3.0	NaN	NaN
2	85	1.0	23.732346		15.0	1.0	1300000.0	10.0
3	30	0.0	25.015243		16.0	3.0	NaN	NaN
4	53	0.0	32.290249		14.0	1.0	NaN	NaN

5 rows × 23 columns

GPAQ : ordinal

El equipo clasificó **GPAQ** como **nominal** en la Fase 1. Este cuaderno corrige esa clasificación a **ordinal**, y conviene mostrar por qué con los datos, no solo afirmarlo.

GPAQ clasifica el nivel de actividad física en **1** Bajo, **2** Moderado, **3** Alto: una escala con un orden real (más actividad física es, en ese sentido, "más" que menos actividad). Si se codificara como nominal —igual que **di3** o **dis2**— con *one-hot*, el modelo vería tres categorías sin relación entre sí y perdería por completo la información de que "Alto" implica más actividad que "Moderado", que a su vez implica más que "Bajo".

In [24]:

```
# Se conservan como enteros ordinales: no se aplica one-hot ni se remapea na
# solo se fija el tipo tras la imputacion de la seccion 4.
df["GPAQ"] = df["GPAQ"].astype(int)
# as28 usa "Int64" (con I mayuscula), el entero especial de pandas que sí ac
# NaN. Un int normal no puede tener NaN adentro: como as28 se dejó con nulos
# proposito (decision e, seccion 4), forzar int comun haria fallar el notebc
df["as28"] = df["as28"].astype("Int64")
df["HTA"] = df["HTA"].astype(int) # ya era 0/1: solo se fija el tipo

df[["GPAQ", "as28", "HTA"]].describe()
```

Out [24]:

	GPAQ	as28	HTA
count	5511.000000	4695.0	5511.000000
mean	1.908728	5.339936	0.362911
std	0.877537	2.435161	0.480883
min	1.000000	1.0	0.000000
25%	1.000000	3.0	0.000000
50%	2.000000	5.0	0.000000
75%	3.000000	7.0	1.000000
max	3.000000	11.0	1.000000

6. Escalamiento (estandarización)

Qué es. Estandarizar (z-score) transforma una variable para que tenga **media 0 y desviación estándar 1**:

$$z = \frac{x - \mu}{\sigma}$$

Qué se escala y qué no. Solo las variables con rol **continua** — Edad , IMC , as27 — están en escalas muy distintas entre sí (años, kg/m², pesos chilenos) y alimentarán eventualmente modelos basados en distancias. No se escalan las binarias, las columnas *one-hot*, las ordinales (GPAQ , as28) ni las variables de diseño muestral (Fexp_F1F2p_Corr , Conglomerado , Estrato): estandarizarlas destruiría su interpretación.

```
In [25]: def escalar_caracteristicas(df, columnas):
        """
        Estandariza (z-score) las columnas indicadas con StandardScaler.

        Retorna el DataFrame transformado y el scaler ajustado, por si hay que a
        la misma transformacion a datos nuevos.
        """
        df = df.copy()
        scaler = StandardScaler()
        df[columnas] = scaler.fit_transform(df[columnas])
        return df, scaler

columnas_continuas = ["Edad", "IMC", "as27"]
df, scaler = escalar_caracteristicas(df, columnas_continuas)
df[columnas_continuas].describe()
```

Out [25]:

	Edad	IMC	as27
count	5.511000e+03	5.511000e+03	4.695000e+03
mean	1.237745e-16	-4.435251e-16	-1.210722e-17
std	1.000091e+00	1.000091e+00	1.000107e+00
min	-1.775999e+00	-2.550353e+00	-7.239530e-01
25%	-8.416285e-01	-6.938584e-01	-4.337133e-01
50%	4.083243e-02	-1.109724e-01	-2.724690e-01
75%	7.675650e-01	5.445377e-01	5.001953e-02
max	2.532487e+00	6.223151e+00	2.504288e+01

Los tres escaladores, comparados

`as27` es precisamente el caso donde esta comparación importa: tiene los atípicos más extremos del conjunto (sección 2).

Escalador	Qué garantiza	Cuándo conviene	Riesgo
StandardScaler	Media 0, desviación 1	Distribución aproximadamente simétrica	No acota el rango
MinMaxScaler	Rango [0, 1]	Se requiere un rango acotado	Un extremo comprime todo el resto
RobustScaler	Mediana 0, RIC 1	Hay valores extremos	Las cifras no quedan «redondas»

```
In [26]: X = df_crudo[["Edad", "IMC", "as27"]].copy()
X["IMC"] = X["IMC"].fillna(X["IMC"].median())
X["as27"] = X["as27"].fillna(X["as27"].median())

resultados = {"original": X}
for nombre, clase in [("standard", StandardScaler), ("minmax", MinMaxScaler)]:
    resultados[nombre] = pd.DataFrame(
        clase().fit_transform(X), columns=X.columns, index=X.index)

for nombre, valores in resultados.items():
    print(f"\n{nombre.upper()}")
    print(valores.describe().round(2).loc[["mean", "50%", "std", "min", "max"]])
```

ORIGINAL

	Edad	IMC	as27
mean	49.20	28.93	438850.83
50%	50.00	28.32	300000.00
std	19.27	5.56	568862.46
min	15.00	14.76	20000.00
max	98.00	63.51	1600000.00

STANDARD

	Edad	IMC	as27
mean	-0.00	-0.00	0.00
50%	0.04	-0.11	-0.24
std	1.00	1.00	1.00
min	-1.78	-2.55	-0.74
max	2.53	6.22	27.36

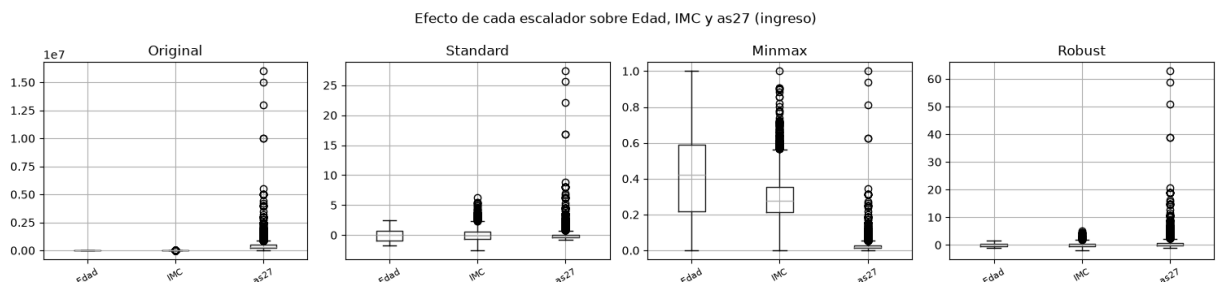
MINMAX

	Edad	IMC	as27
mean	0.41	0.29	0.03
50%	0.42	0.28	0.02
std	0.23	0.11	0.04
min	0.00	0.00	0.00
max	1.00	1.00	1.00

ROBUST

	Edad	IMC	as27
mean	-0.03	0.09	0.56
50%	0.00	0.00	0.00
std	0.62	0.81	2.28
min	-1.13	-1.97	-1.12
max	1.55	5.10	62.80

```
In [27]: fig, ejes = plt.subplots(1, 4, figsize=(15, 3.6))
for eje, (nombre, valores) in zip(ejes, resultados.items()):
    valores.boxplot(ax=eje)
    eje.set_title(nombre.capitalize())
    eje.tick_params(axis="x", rotation=30, labelsiz=8)
plt.suptitle("Efecto de cada escalador sobre Edad, IMC y as27 (ingreso)")
plt.tight_layout()
plt.show()
```



```
In [28]: def verificar_escalamiento(valores, metodo, tol=1e-9):
    """Comprueba que el escalamiento produjo lo que el metodo promete."""
    if metodo == "standard":
        assert np.allclose(valores.mean(), 0, atol=tol), "media distinta de 0"
        assert np.allclose(valores.std(ddof=0), 1, atol=tol), "desviacion distinta de 1"
        return "media ~ 0 y desviacion ~ 1"
```

```

if metodo == "minmax":
    assert np.allclose(valores.min(), 0, atol=tol), "minimo distinto de
    assert np.allclose(valores.max(), 1, atol=tol), "maximo distinto de
    return "todos los valores en el rango [0, 1]"
if metodo == "robust":
    assert np.allclose(valores.median(), 0, atol=1e-8), "mediana distinta
    ric = valores.quantile(0.75) - valores.quantile(0.25)
    assert np.allclose(ric, 1, atol=1e-8), "rango intercuartilico distinto
    return "mediana ~ 0 y rango intercuartilico ~ 1"
raise ValueError(f"Metodo desconocido: {metodo}")

for metodo in ["standard", "minmax", "robust"]:
    print(f"[OK] {metodo:<9} {verificar_escalamiento(resultados[metodo]['as27'])}")

```

[OK] standard media ~ 0 y desviacion ~ 1
[OK] minmax todos los valores en el rango [0, 1]
[OK] robust mediana ~ 0 y rango intercuartilico ~ 1

Coherencia entre decisiones — queda abierta. `as27` se imputó mputación parcial por tramo de `as28` para 177 casos; ese mismo hecho apuntaría a usar `RobustScaler` en vez de `StandardScaler` para esa variable en particular. Este cuaderno usa `StandardScaler` para las tres continuas por consistencia con `Edad` e `IMC` (que no muestran esa asimetría tan marcada), pero la decisión de tratar `as27` con un escalador distinto queda como algo a resolver en la Fase 3, no algo resuelto aquí sin más.

7. Validación técnica y verificación del código

`validar_dataset` comprueba, con `assert`, que el resultado es correcto:

1. **Integridad** — sin valores nulos fuera de los documentados como faltante genuino (`as27`, `as28`; ver sección 4, decisiones c y e).
2. **Consistencia** — todas las columnas son numéricas.
3. **Coherencia** — cada grupo *one-hot* suma exactamente 1 por fila.
4. **Duplicados** — cuántas filas repetidas hay.
5. **Dimensiones** — la forma final de la tabla.

In [29]:

```

def validar_dataset(df, grupos_one_hot=None, nulos_esperados=None):
    """
    Comprueba integridad, consistencia y coherencia del dataset final.
    Lanza AssertionError si alguna comprobacion falla.

    Parametros
    -----
    nulos_esperados : dict, opcional
        {columna: cantidad esperada de nulos} para columnas que se dejaron
        con faltantes genuinos a proposito (seccion 4). Cualquier nulo fuera
    """

```

de estas columnas, o una cantidad distinta a la esperada, hace fallar la validacion: no es "permitir nulos", es verificar que son exactamente los documentados.

"""

```
print("VALIDACION DEL DATASET FINAL")
print("-" * 45)
```

```
nulos_esperados = nulos_esperados or {}
nulos_por_columna = df.isnull().sum()
nulos_inesperados = {
    col: int(n) for col, n in nulos_por_columna.items()
    if n > 0 and n != nulos_esperados.get(col, 0)
}
assert not nulos_inesperados, f"Nulos no esperados o en cantidad distinta"
for col, n_esperado in nulos_esperados.items():
    n_real = int(nulos_por_columna.get(col, 0))
    assert n_real == n_esperado, f"'{col}': se esperaban {n_esperado} nulos"
total_nulos = int(nulos_por_columna.sum())
if nulos_esperados:
    print(f"[OK] {total_nulos} nulos en total, todos documentados como esperados")
else:
    print(f"[OK] Sin valores nulos (total = {total_nulos})")

no_numericas = df.select_dtypes(exclude=[np.number]).columns.tolist()
assert not no_numericas, f"Columnas no numericas: {no_numericas}"
print("[OK] Todas las columnas son numericas")
```

```
if grupos_one_hot:
    for nombre, cols in grupos_one_hot.items():
        suma = df[cols].sum(axis=1)
        assert (suma == 1).all(), f"El grupo '{nombre}' no suma 1 en todas las filas"
    print(f"[OK] Grupo one-hot '{nombre}' coherente")
```

```
dup = int(df.duplicated().sum())
print(f"[INFO] Filas duplicadas: {dup}")
print(f"[INFO] Dimensiones finales: {df.shape}")
return True
```

```
grupos = {
    "Sexo": ["Sexo_Hombre", "Sexo_Mujer"],
    "Zona": ["Zona_Urbano", "Zona_Rural"],
    "di3": ["di3_Si", "di3_No", "di3_No_recuerda"],
    "dis2": ["dis2_Si_una_vez", "dis2_Si_mas_de_una_vez", "dis2_Nunca", "dis2_Nunca_recuerda"]
}

# as27 y as28 se dejaron con 816 nulos a proposito (seccion 4, decisiones c)
# es la misma no respuesta economica en ambas, documentada, no un descuido.
nulos_esperados = {"as27": 816, "as28": 816}
validar_dataset(df, grupos, nulos_esperados)
```

VALIDACION DEL DATASET FINAL

```
[OK] 1632 nulos en total, todos documentados como faltante genuino: {'as27': 816, 'as28': 816}
[OK] Todas las columnas son numericas
[OK] Grupo one-hot 'Sexo' coherente
[OK] Grupo one-hot 'Zona' coherente
[OK] Grupo one-hot 'di3' coherente
[OK] Grupo one-hot 'dis2' coherente
[INFO] Filas duplicadas: 0
[INFO] Dimensiones finales: (5511, 23)
```

Out[29]: True

Sobre "no aplica". Ninguna de las seis variables revisadas depende de un salto condicional del cuestionario (una pregunta que solo se hace si la persona respondió algo específico antes): `as27`, `as28`, `HTA`, `di3`, `dis2` y `GPAQ` se preguntan a toda persona encuestada dentro de su formulario correspondiente. Por eso no existe, para estas variables, la categoría "no aplica" que menciona el libro de códigos para otras preguntas del cuestionario — y no corresponde inventarla ni tratarla como si estuviera presente.

Pruebas de las funciones (casos normal, límite y excepciones)

Además de validar el *dataset*, se prueban por separado las funciones nuevas de este cuaderno en tres escenarios: caso normal, y las dos excepciones que cada una declara.

```
In [30]: # --- codificar_one_hot: caso normal ---
prueba = pd.DataFrame({"color": ["rojo", "azul", "rojo", "verde"]})
res = codificar_one_hot(prueba, "color", ["azul", "rojo", "verde"])
assert res.shape == (4, 3), "Numero de columnas inesperado."
assert (res.sum(axis=1) == 1).all(), "Cada fila debe tener una sola categoría"
print("[OK] codificar_one_hot: caso normal")

# --- codificar_one_hot: excepciones ---
try:
    codificar_one_hot(prueba, "color", ["azul", "rojo"])
except ValueError as e:
    print(f"[OK] codificar_one_hot: ValueError capturado ({e})")

try:
    codificar_one_hot(prueba, "inexistente", ["x"])
except KeyError as e:
    print(f"[OK] codificar_one_hot: KeyError capturado ({e})")

# --- marcar_codigos_no_respuesta: caso normal y limite ---
prueba2 = pd.DataFrame({"ingreso": [100, -9999, 200, -8888, 300]})
limpio = marcar_codigos_no_respuesta(prueba2, "ingreso")
assert limpio["ingreso"].isna().sum() == 2, "Debian marcarse 2 codigos especiales"
print("[OK] marcar_codigos_no_respuesta: caso normal")

sin_codigos = pd.DataFrame({"ingreso": [100, 200, 300]})
resultado_sin_cambios = marcar_codigos_no_respuesta(sin_codigos, "ingreso")
```

```

assert resultado_sin_cambios["ingreso"].isna().sum() == 0, "No debia marcar
print("[OK] marcar_codigos_no_respuesta: caso limite (sin codigos presentes)

try:
    marcar_codigos_no_respuesta(prueba2, "inexistente")
except KeyError as e:
    print(f"[OK] marcar_codigos_no_respuesta: KeyError capturado ({e})")

# --- imputar_nulos_categoricos: caso normal y excepcion ---
prueba3 = pd.DataFrame({"nivel": [1, 2, 2, np.nan, 2, 1]})
imputado = imputar_nulos_categoricos(prueba3, "nivel", estrategia="moda")
assert imputado["nivel"].isna().sum() == 0, "No debian quedar nulos."
assert imputado.loc[3, "nivel"] == 2, "Debia imputarse con la moda (2)."
print("[OK] imputar_nulos_categoricos: caso normal")

try:
    imputar_nulos_categoricos(prueba3, "nivel", estrategia="media")
except ValueError as e:
    print(f"[OK] imputar_nulos_categoricos: ValueError capturado ({e})")

```

```

[OK] codificar_one_hot: caso normal
[OK] codificar_one_hot: ValueError capturado (Se esperaban 3 nombres para 'c
olor' (['azul', 'rojo', 'verde']), pero se recibieron 2.)
[OK] codificar_one_hot: KeyError capturado ("La columna 'inexistente' no exi
ste en el DataFrame.")
'ingreso': 2 valores con codigo de no respuesta marcados como NaN
[OK] marcar_codigos_no_respuesta: caso normal
'ingreso': 0 valores con codigo de no respuesta marcados como NaN
[OK] marcar_codigos_no_respuesta: caso limite (sin codigos presentes)
[OK] marcar_codigos_no_respuesta: KeyError capturado ("La columna 'inexisten
te' no existe en el DataFrame.")
'nivel': 1 nulos imputados con la moda = 2.0
[OK] imputar_nulos_categoricos: caso normal
[OK] imputar_nulos_categoricos: ValueError capturado (estrategia debe ser 'm
oda')

```

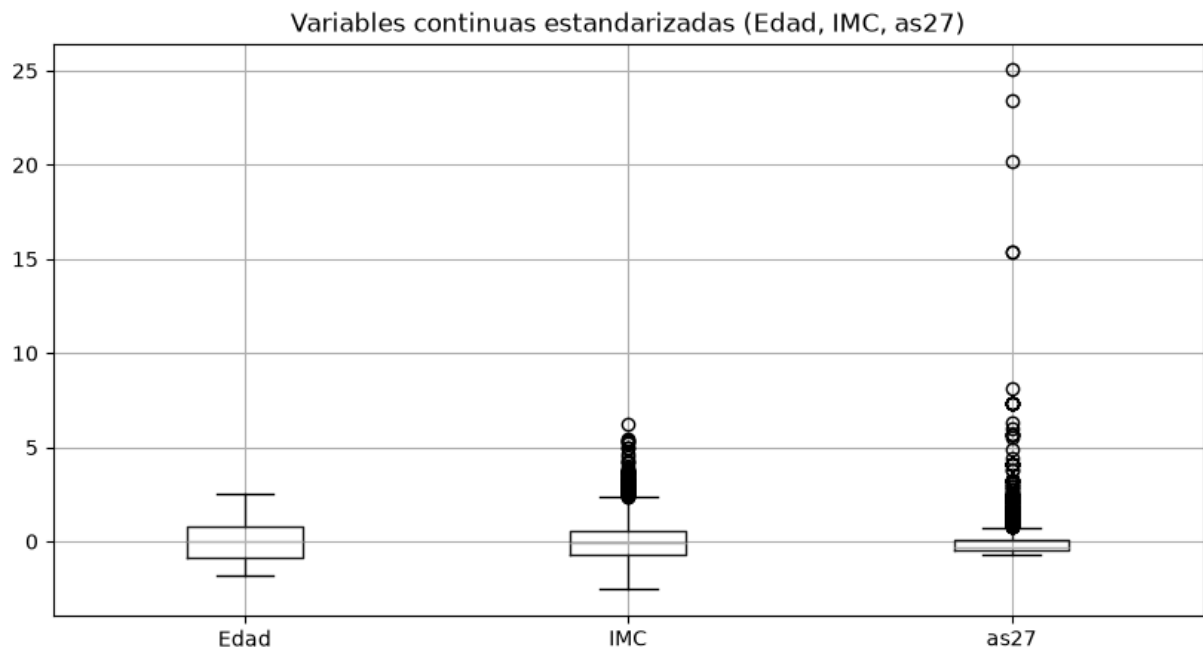
Visualización de control

El *boxplot* de las tres variables continuas ya estandarizadas confirma que quedaron centradas en torno a 0 y en una escala comparable entre sí.

```

In [31]: df[columnas_continuas].boxplot(figsize=(10, 5))
plt.title("Variables continuas estandarizadas (Edad, IMC, as27)")
plt.show()

```



Vista final del *dataset* completamente preprocesado:

```
In [32]: print("Dataset final:", df.shape)
df.head()
```

Dataset final: (5511, 23)

```
Out[32]:
```

	Edad	HTA	IMC	anos_estudio_MINSAL_1	GPAQ	as27	as28	Fexr
0	-1.256904	0	-1.031300	15.0	3	-0.433713	3	
1	-1.516452	0	-0.490394	14.0	3	NaN	<NA>	
2	1.857664	1	-0.935179	15.0	1	1.339974	10	
3	-0.997357	0	-0.704314	16.0	3	NaN	<NA>	
4	0.196561	0	0.604864	14.0	1	NaN	<NA>	

5 rows x 23 columns

Conclusiones y trazabilidad

El *pipeline* dejó el subconjunto de trabajo sin valores nulos, con todas las variables en formato numérico, las binarias y nominales codificadas, las ordinales conservando su orden, y las continuas estandarizadas. Cada paso se implementó como una función documentada y reutilizable (`seleccionar_variables_ens` , `filtrar_ponderador_valido` , `explorar_dataframe` , `marcar_codigos_no_respuesta` , `imputar_nulos_numericos` , `imputar_nulos_categoricos` , `codificar_one_hot` , `escalar_caracteristicas` , `validar_dataset`).

Trazabilidad con el repositorio (F2): este cuaderno vive en `F2/notebooks/` y regenera, siempre desde `data/raw/ens2016.xlsx`, los mismos archivos que `F1/notebooks/S1_F1_Definicion.ipynb` reconoce al auditar la estructura del repositorio. El `README` documenta las dependencias y las instrucciones de ejecución.

8. Recursividad: aplanar los metadatos del proyecto

Todo el procesamiento anterior fue **estructurado**: secuencias, condicionales y bucles. Esta sección usa **recursividad**, la herramienta natural cuando la estructura tiene profundidad desconocida.

El caso es real: los metadatos del proyecto forman un diccionario anidado y, para exportarlos como tabla, hay que convertirlos en pares clave-valor. Un bucle no sirve porque no se sabe de antemano cuántos niveles de anidamiento tiene.

Anatomía de la función recursiva:

- *Caso base*: el valor no es un diccionario → se devuelve el par clave-valor.
- *Caso recursivo*: el valor es un diccionario → la función se llama a sí misma un nivel más abajo, arrastrando el prefijo de la ruta.

```
In [33]: METADATOS = {
    "proyecto": {
        "titulo": "Factores sociodemograficos y riesgo cardiovascular",
        "fase": "F2",
        "semilla": SEMILLA,
    },
    "datos": {
        "archivo_origen": "ens2016.xlsx",
        "fuente": {
            "institucion": "MINSAL – Departamento de Epidemiologia",
            "encuesta": "ENS 2016-2017",
        },
        "ponderador": "Fexp_F1F2p_Corr",
        "n_filas_final": int(df.shape[0]),
    },
    "decisiones": {
        "codigos_no_respuesta": {"as28": "-9999 marcado como NaN"},
        "filas_eliminadas": {"HTA_nulo": 9},
        "imputacion": {
            "IMC": "mediana",
            "anos_estudio_MINSAL_1": "mediana",
            "GPAQ": "moda",
            "as27_parcial": "mediana por tramo de as28 (177 casos sin monto",
        },
        "faltantes_genuinos_no_imputados": {
            "as27": "816 nulos (misma no respuesta economica que as28, no al",
            "as28": "816 nulos (no respuesta de ingreso; ver as28_no_responc
```

```

    },
    "reclasificacion": {"GPAQ": "de nominal (equipo) a ordinal (este cua
    "escalador": "standard",
    "variables_no_transformadas": ["Fexp_F1F2p_Corr", "Conglomerado", "E
    },
}

def aplanar(estructura, prefijo="", separador="."):
    """Aplana un diccionario anidado en uno de un solo nivel.

    Ejemplo
    -----
    >>> aplanar({"a": {"b": 1, "c": {"d": 2}}})
    {'a.b': 1, 'a.c.d': 2}
    """
    plano = {}
    for clave, valor in estructura.items():
        ruta = f"{prefijo}{separador}{clave}" if prefijo else str(clave)
        if isinstance(valor, dict):
            plano.update(aplanar(valor, ruta, separador))          # CASO RE
        elif isinstance(valor, (list, tuple)):
            plano[ruta] = ", ".join(str(v) for v in valor)
        else:
            plano[ruta] = valor                                     # CASO E
    return plano

metadatos_planos = aplanar(METADATOS)
print(f"El diccionario anidado se aplano en {len(metadatos_planos)} pares.\r
for clave, valor in metadatos_planos.items():
    print(f" {clave:<38} {str(valor)[:52]}")

```

El diccionario anidado se aplano en 19 pares.

proyecto.titulo	Factores sociodemograficos y riesgo cardiovascular
proyecto.fase	F2
proyecto.semilla	2026
datos.archivo_origen	ens2016.xlsx
datos.fuente.institucion	MINSAL – Departamento de Epidemiologia
datos.fuente.encuesta	ENS 2016–2017
datos.ponderador	Fexp_F1F2p_Corr
datos.n_filas_final	5511
decisiones.codigos_no_respuesta.as28	–9999 marcado como NaN
decisiones.filas_eliminadas.HTA_nulo	9
decisiones.imputacion.IMC	mediana
decisiones.imputacion.anos_estudio_MINSAL_1	mediana
decisiones.imputacion.GPAQ	moda
decisiones.imputacion.as27_parcial	mediana por tramo de as28 (177 casos sin monto, con
decisiones.faltantes_genuinos_no_imputados.as27	816 nulos (misma no respuesta economica que as28, no
decisiones.faltantes_genuinos_no_imputados.as28	816 nulos (no respuesta de ingreso; ver as28_no_resp
decisiones.reclasificacion.GPAQ	de nominal (equipo) a ordinal (este cuaderno)
decisiones.escalador	standard
decisiones.variables_no_transformadas	Fexp_F1F2p_Corr, Conglomerado, Estrato

```
In [34]: assert aplanar({"a": 1}) == {"a": 1}, "Caso plano"
assert aplanar({"a": {"b": 1}}) == {"a.b": 1}, "Un nivel de anidamiento"
assert aplanar({"a": {"b": {"c": {"d": 4}}}}) == {"a.b.c.d": 4}, "Anidamiento"
assert aplanar({}) == {}, "Caso limite: diccionario vacio"
assert aplanar({"x": [1, 2, 3]}) == {"x": "1, 2, 3"}, "Listas convertidas a string"
print("[OK] La funcion recursiva pasa las cinco pruebas.")
```

[OK] La funcion recursiva pasa las cinco pruebas.

Cuándo NO usar recursividad. Si la profundidad se conoce y es fija, un bucle anidado es más claro y más rápido. Se justifica aquí porque la profundidad de `METADATOS` es variable.

9. Persistencia y trazabilidad

El resultado debe quedar guardado y documentado: es la entrada directa de la Fase 3.

```
In [35]: ARCHIVO_PROCESADO = DIR_PROCESADO / "ens_procesado.csv"
df.to_csv(ARCHIVO_PROCESADO, index=False)

# Verificacion de ida y vuelta: se relee lo guardado y se comprueba que coincida
releido = pd.read_csv(ARCHIVO_PROCESADO)
```

```

assert releido.shape == df.shape, "El archivo releido no coincide en dimensi
print(f"Guardado y verificado: {ARCHIVO_PROCESADO}")
print(f"Tamano en disco: {ARCHIVO_PROCESADO.stat().st_size / 1024:.1f} kB")

pd.DataFrame(list(metadatos_planos.items()), columns=["clave", "valor"]).to_
DIR_DOCS / "metadatos_proyecto.csv", index=False)
print(f"Metadatos exportados a {DIR_DOCS / 'metadatos_proyecto.csv'}")

```

Guardado y verificado: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo
 4-mcdi500/data/processed/ens_procesado.csv
 Tamano en disco: 656.9 kB
 Metadatos exportados a /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo
 4-mcdi500/F2/docs/metadatos_proyecto.csv

```

In [36]: resumen = pd.DataFrame([
    {"indicador": "Filas", "inicial": df_crudo.shape[0], "final": df.shape[0]},
    {"indicador": "Columnas", "inicial": df_crudo.shape[1], "final": df.shape[1]},
    {"indicador": "Valores nulos",
     "inicial": int(df_crudo.isna().sum().sum()), "final": int(df.isna().sum().sum())},
    {"indicador": "Columnas no numericas",
     "inicial": int(df_crudo.select_dtypes(exclude=[np.number]).shape[1]),
     "final": int(df.select_dtypes(exclude=[np.number]).shape[1])},
    {"indicador": "Filas duplicadas",
     "inicial": int(df_crudo.duplicated().sum()), "final": int(df.duplicated().sum())}
])
resumen

```

```

Out[36]:

```

	indicador	inicial	final
0	Filas	5520	5511
1	Columnas	16	23
2	Valores nulos	1284	1632
3	Columnas no numericas	1	0
4	Filas duplicadas	0	0

10. Reflexión técnica

Hallazgos

- **Solo as28 usa realmente códigos de no respuesta.** De las seis variables candidatas (as27 , as28 , di3 , di2 , HTA , GPAQ), únicamente as28 contenía el código -9999 ; -8888 no aparece en ninguna. Verificarlo evitó aplicar un tratamiento innecesario a variables que no lo requerían.
- **as27 y as28 comparten la misma no respuesta económica, y eso cambió cómo se tratan.** Cruzar ambas variables mostró que los 816 casos sin dato en as28 (código -9999) son exactamente los mismos que quedan sin dato en

`as27` : no son dos fallas independientes. Imputar cualquiera de las dos con un valor típico (moda o mediana) habría escondido un posible sesgo de no respuesta concentrado en los extremos de ingreso. Se dejan como faltante genuino donde el patrón está verificado (esos 816 casos en ambas variables), y se imputa solo donde hay información real disponible: los otros 177 nulos de `as27` sí tienen un tramo de `as28` conocido, así que se imputan con la mediana dentro de ese tramo. El mismo criterio se evaluó para `GPAQ` (también ordinal, también con nulos) y se descartó: ahí la duda sobre el mecanismo es especulativa, no verificada, y el costo de imputar mal es bajo (3,6 % de nulos), así que se mantiene la imputación por moda.

- **`GPAQ` estaba mal clasificado.** El equipo lo declaró "nominal" en la Fase 1, pero es una escala ordinal (Bajo < Moderado < Alto). Tratarlo como nominal con *one-hot* habría descartado esa información de orden sin necesidad.

Dificultades y cómo se resolvieron

- **Rutas relativas frágiles.** La versión anterior de este cuaderno mezclaba rutas relativas al directorio de trabajo (`data/raw`) con búsquedas hacia arriba (`Path.cwd().parent`), lo que dependía de desde dónde se abriera Jupyter. Se reemplazó por `encontrar_raiz_proyecto()` — la misma función que ya usa la Fase 1 — para que el cuaderno corra igual sin importar la carpeta de trabajo.
- `di3` y `dis2` con categorías de incertidumbre. Sus categorías minoritarias se verificaron contra el libro de códigos F1: `di3=3` y `dis2=4` significan "No recuerda", una respuesta de incertidumbre real, no un vacío ni una respuesta negativa. Se codificaron con su nombre real (`di3_No_recuerda` , `dis2_No_recuerda`) en vez de fusionarlas con las categorías negativas.

Decisiones que quedan abiertas

- Si `as27` debería escalarse con `RobustScaler` en vez de `StandardScaler` , dada su asimetría.
- Si el análisis de la Fase 3 usará `Conglomerado` , `Estrato` y `Fexp_F1F2p_Corr` para un análisis con diseño muestral complejo, o solo el ponderador de forma simple.
- Si vale la pena extender `comparar_imputaciones()` para soportar un agrupador ya categórico (sin `qcut`), y así comparar con cifras "mediana por edad" vs. "mediana por tramo de `as28` " para los 177 casos — hoy esa elección se argumenta en sección 2 pero no se demuestra con una tabla, a diferencia del resto de las decisiones de este cuaderno.

Declarar lo que queda abierto evidencia más dominio que presentar todo resuelto.

11. Verificación antes de entregar

Notebook

- Corre completo tras *Kernel* → *Restart Kernel and Run All Cells*, sin errores.
- La numeración de ejecución es continua.
- Cada bloque de código está precedido por una celda narrativa que explica su lógica.
- Todas las rutas se resuelven desde la raíz del repositorio
(`encontrar_raiz_proyecto`) y la semilla está declarada.

Contenido técnico

- La procedencia del conjunto está documentada, con cita en APA 7.
- El diccionario de variables declara el rol analítico de cada una y documenta la reclasificación de `GPAQ`.
- Los códigos especiales de no respuesta se verificaron por variable, no se asumieron.
- Las decisiones de imputación y escalamiento están comparadas y justificadas con cifras del propio conjunto.
- Los faltantes genuinos que quedan a propósito (`as27` , `as28` : 816 cada una) están documentados y verificados por `validar_dataset()` , no son un descuido.
- Hay pruebas de casos normales, límite y excepciones para cada función nueva.

Archivos

- `data/raw/ens2016.xlsx` conserva el archivo original sin modificar.
- `data/processed/` contiene `ens_variables_seleccionadas.xlsx` , `ens_variables_f1f2.xlsx` y `ens_procesado.csv` (entrada directa de la Fase 3).
- `F2/docs/` contiene el diccionario de variables, el resumen del filtro y los metadatos.

Repositorio

- Carpeta `F2/notebooks/` con este cuaderno, y `README` actualizado con la estructura vigente.